Deep Reinforcement Learning

Advanced Algorithms (Part I)

Prof. Dr. Sebastian Peitz

Chair of Safe Autonomous Systems, TU Dortmund

Summer term 2026

Content

Content

- The evolution of modern RL algorithms
- Part (I): Improving policy gradient methods
  - Natural policy gradient
  - Trust-region policy optimization (TRPO)
  - Proximal policy optimization (PPO)

Where are we?

Chapter	Topic	Content
	Basics & tabular methods	
1-5	Bandits, MDPs, Dynamic Programming, Monte Carlo, TD Learning	RL basics in finite dimensions
	Deep-learning-based methods	
6	Brief introduction to deep learning	The basics for what comes next
7	Value function approximation	Value estimation with function approximation
8	Deep Q-learning	Q-learning with neural networks
9	Policy gradients	Direct optimization of the policy
10	Actor-critic algorithms	Improved policy gradients via value functions
11	Advanced algorithms (Part I): From policy gradient to PPO	The evolution of modern RL algorithms
12	Advanced algorithms (Part II): From Q-learning to Soft Actor-Critic	
13	Exploration	
	Model-Based Control	
	Advanced Topics	

Chapter	Topic	Content

Lecture contents

The evolution of modern RL algorithms

The evolution of modern RL algorithms

(I) Policy gradient methods

- Policy is explicitly parameterized and optimized directly:

$$L_\pi(\phi) = \mathbb{E}_{\tau \sim p_\phi(\tau)}\left[r(\tau)\right].$$

- Gradient ascent: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.
- Methods are typically **on-policy**.
- **Algorithms**: REINFORCE → Actor-Critic → Natural Policy Gradient → Trust-region policy optimization (TRPO) → Proximal policy optimization (PPO).

Shortcomings

- High variance gradients.
- Unstable policy updates.
- Catastrophic performance collapse.

Improvement strategy

- How to safely update policies.

(II) Value-based methods

- Approximation of the Q-function:

$$Q^*(s,a) = r + \max_{a' \in \mathcal{A}} Q^*(s',a').$$

- Extract implicit policy: $\pi(s) = \arg\max_{a \in \mathcal{A}} Q^*(s,a)$.
- Typically **off-policy**.
- **Algorithms**: Q-learning → DQN → Deep deterministic policy gradient (DDPG) → TD3 → Soft actor-critic (SAC).

Shortcomings

- Instability from bootstrapping.
- Overestimation bias.
- Maximization over actions / continuous action spaces.

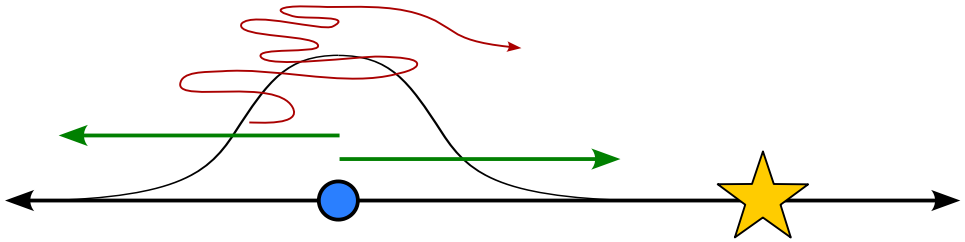
Improvement strategy

- Stabilizing Q-learning with function approximation.

- Solving the continuous argmax problem.

Part (I): Improving policy gradient methods

Ill-conditioned gradients

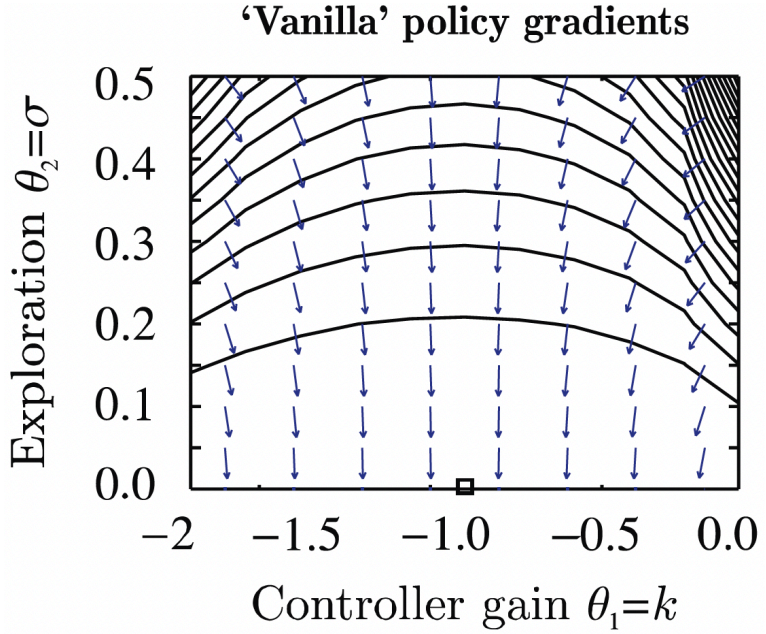


A simple exemplary MDP (Peters and Schaal 2008). Inspired by Sergey Levine's CS285 lecture.

$$r_t = -s_t^2 - a_t^2$$

$$\log \pi_\phi(a_t|s_t) = -\frac{1}{2\sigma^2}(ks_t - a_t)^2 + C, \qquad \phi = (k, \sigma).$$

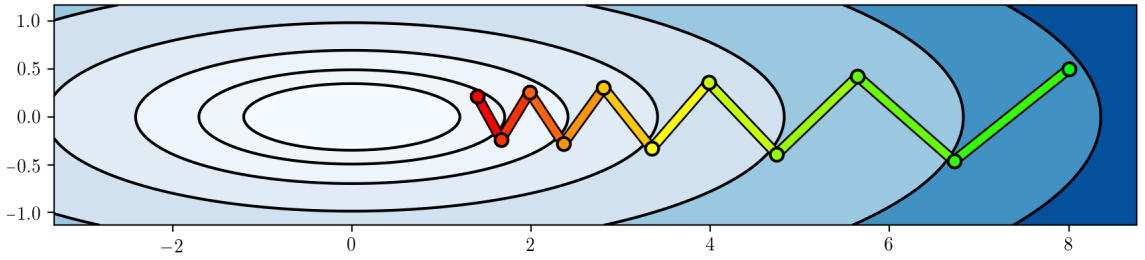
$$\nabla_\phi \log \pi_\phi(a_t|s_t) = \begin{pmatrix} -\frac{(ks_t - a_t)s_t}{\sigma^2} \\ \frac{(ks_t - a_t)^2}{\sigma^3} \end{pmatrix}$$



Source: (Peters and Schaal 2008)

- **Optimum**: $\phi^* = (-1, 0)$.
- **Issue**: The gradients do not point towards the optimum for smaller σ!
  - The σ^{-3} term blows up.

Closely related to ill-conditioned optimization problems:



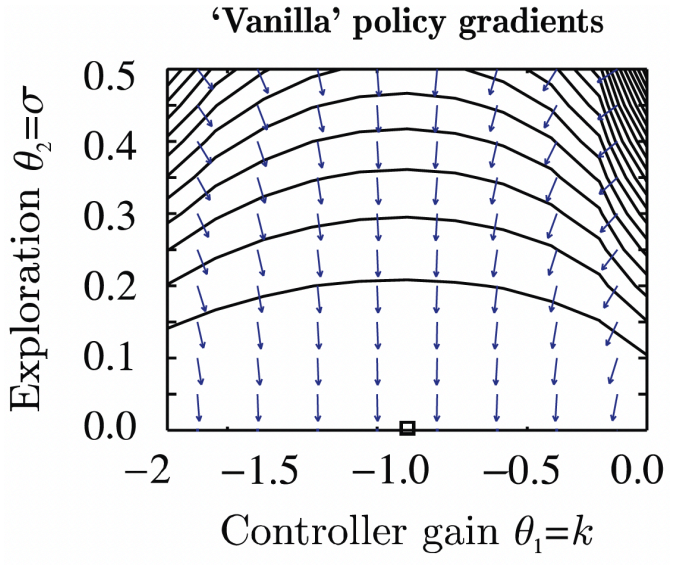
Constraining the ascent direction

- Recall the policy gradient update: $\phi \leftarrow \phi + \alpha \nabla_\phi L_\pi(\phi)$.
- A natural idea: **Constrain the length of our update step**!
  - Linearization of our optimization problem around ϕ using Taylor series expansion:

$$L(\phi') = L_\pi(\phi) + \sum_{i=1}^{n} \frac{\partial L}{\partial \phi_i}(\phi'_i - \phi_i) + \mathcal{O}((\phi' - \phi)^2)$$
$$= L_\pi(\phi) + (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi).$$

  - Constrained optimization of ϕ' along the steepest ascent direction:

$$\phi' \leftarrow \arg\max_{\phi'} (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi) \qquad \text{subject to} \qquad \|\phi' - \phi\|^2 \leq \epsilon.$$



Source: (Peters and Schaal 2008)

- **But**: We just saw that some parameters change probabilities *a lot more than others*!
- **Alternative**: Rescale the gradient so that this does not happen!
- A better way to constrain the update distance: The **change in distribution** via the **Kullback-Leibler (KL) divergence**!

$$D_{\text{KL}}\left(\pi_\phi \| \pi'_\phi\right).$$

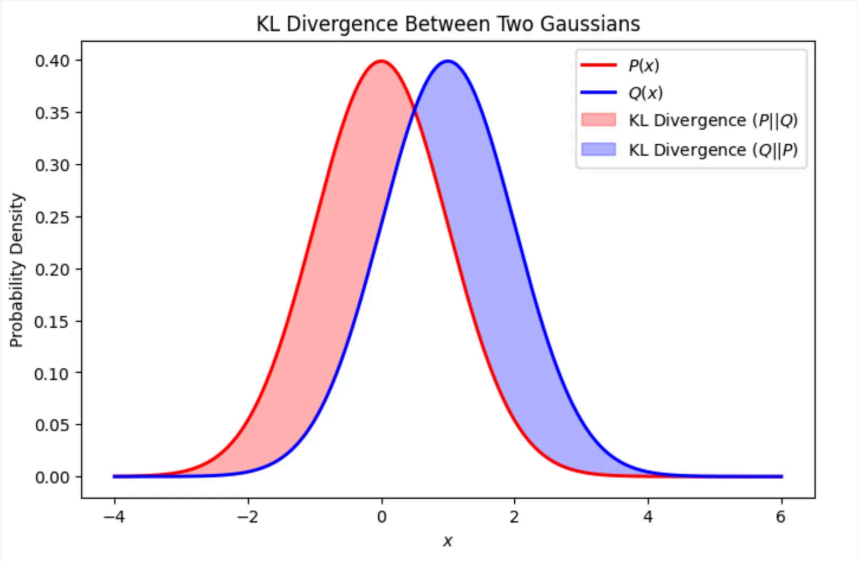
Aside: KL-divergence and Fisher information matrix

Kullback-Leibler divergence

- The **Kullback-Leibler divergence** (or **KL divergence**) measures the "distance" between two distributions (e.g., $p(x)$ and $q(x)$)

$$D_{\mathrm{KL}}(p\|q) = \begin{cases} \sum_{x\in\mathcal{X}} p(x)\log\left(\frac{p(x)}{q(x)}\right) & \text{if } \mathcal{X} \text{ is finite} \\ \int_{\mathcal{X}} p(x)\log\left(\frac{p(x)}{q(x)}\right)\mathrm{d}x & \text{if } \mathcal{X} \text{ is continuous} \end{cases}.$$

  - It is zero if and only if $p(x) = q(x)$.



[Source].

- **Intuition**: Imagine you have two different maps of the exact same city.
  - Map A is completely accurate. Every street, etc., is exactly where it should be.
  - Map B was drawn from memory by a friend. It's mostly right with some minor flaws.
  - If you use Map B to navigate the city, you are going to make some mistakes.
  - The KL divergence answers the question: "How much extra 'gas' (or information) will I waste if I use Map B instead of Map A?"
- KL Divergence measures the *surprise* or *unfair penalty* you get when you use q to predict something that actually follows p. If q is a perfect match for p (i.e., $q = p$), your surprise is zero. The worse your guess (q) is, the higher the KL divergence.
- **Question**: What happens if our "model" q is absolutely certain that something is **not** going to happen ($q(x) = 0$ for some x)?

💡 The KL divergence is not a real distance. For instance, it it not symmetric (i.e., $D_{\mathrm{KL}}(p\|q) \neq D_{\mathrm{KL}}(q\|p)$ in general), which means that it also does not satisfy the triangle inequality. Still, it gives us a measure of how far two distributions are apart.

Fisher information matrix

- The **Fisher information matrix** (**FIM**) measures how much information an observable random variable x carries about an unknown parameter ϕ (e.g., the weights of a neural network).
- Mathematically, if we have a log-likelihood function $\log p(x|\phi)$, the FIM (denoted as F) is the variance of its gradient (the "*score*"):

$$F = \mathbb{E}\left[(\nabla_\phi \log p\left(x|\phi\right))(\nabla_\phi \log p(x|\phi))^\top\right]. \tag{1}$$

> **In simpler terms**: How do variations of ϕ influence the resulting probability distribution?
>
> - High Fisher Information: Small change in $\phi \Rightarrow$ large shift in the distribution.
> - Low Fisher Information: Change in $\phi \Rightarrow$ only small shift in the distribution.

Relation to the KL divergence

- Consider a very small change in ϕ by $\Delta\phi$, and study the KL divergence $D_{\mathsf{KL}}\left(p\left(x|\phi\right)\|p\left(x|\phi+\Delta\phi\right)\right)$.
- If we perform a Taylor series expansion of D_{KL} around $\Delta\phi = 0$, then the first two terms are zero ($D_{\mathsf{KL}}\left(p\left(x|\phi\right)\|p\left(x|\phi\right)\right) = 0$ and since $\Delta\phi = 0$ is the minimum, the first derivative is zero as well).
- The second-order term is thus the leading term! $\Rightarrow$ for small $\Delta\phi$, we have

$$D_{\mathsf{KL}}\left(p\left(x|\phi\right)\|p\left(x|\phi+\Delta\phi\right)\right) \approx \frac{1}{2}\Delta\phi^\top F \Delta\phi. \tag{2}$$

KL-divergence and Fisher information in RL

- When using the KL divergence or the Fisher information matrix in RL, we often do so in terms of different policies.
- This means that we consider the difference in the **action distribution** $D_{\mathsf{KL}}\left(\pi_\phi \| \pi_{\phi'}\right)$.
- However, this expression is incomplete or even misleading.
- Instead, we consider
  - for a fixed s,

$$D_{\mathsf{KL}}\left(\pi_\phi(\cdot|s) \| \pi_{\phi'}(\cdot|s)\right) = \int_{\mathcal{A}} \pi_\phi(a|s) \log\left(\frac{\pi_\phi(a|s)}{\pi_{\phi'}(a|s)}\right) \mathrm{d}a,$$

  - the expectation over the state space, i.e.,

$$\bar{D}_{\mathsf{KL}}\left(\pi_\phi \| \pi_{\phi'}\right) = \mathbb{E}_{s\sim p_\phi}\left[D_{\mathsf{KL}}\left(\pi_\phi(\cdot|s) \| \pi_{\phi'}(\cdot|s)\right)\right],$$

  - or the maximum entry, i.e.,

$$D_{\mathsf{KL}}^{\max}\left(\pi_\phi \| \pi_{\phi'}\right) = \max_{s\in\mathcal{S}} D_{\mathsf{KL}}\left(\pi_\phi(\cdot|s) \| \pi_{\phi'}(\cdot|s)\right).$$

- The relation between KL divergence and FIM for small changes can be derived analogously to the previous slide and reads

$$\mathbb{E}_{s\sim p_\phi}\left[D_{\mathsf{KL}}\left(\pi_\phi \| \pi_{\phi'}\right)\right] \approx \frac{1}{2}\Delta\phi^\top \Big(\underbrace{\mathbb{E}_{s\sim p_\phi}[F(s)]}_{=F} \Big)\Delta\phi = \frac{1}{2}\Delta\phi^\top F \Delta\phi.$$

Natural policy gradient

Back to constrained ascent directions

- Recall: constraining w.r.t. the parameter ϕ does not solve the issue of strongly different impacts of individual parameters:

$$\phi' \leftarrow \arg\max_{\phi'} (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi) \qquad \text{subject to} \qquad \|\phi' - \phi\|^2 \leq \epsilon.$$

- **Question**: What is a good alternative to avoid too large (and thus, harmful) steps?
  $\Rightarrow$ the KL divergence!

$$\phi' \leftarrow \arg\max_{\phi'} (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi) \qquad \text{subject to} \qquad \mathbb{E}_{s\sim p_\phi}\left[D_{\mathsf{KL}}\left(\pi_{\phi'}(\cdot|s) \| \pi_\phi(\cdot|s)\right)\right] \leq \epsilon.$$

  - Constraining the KL divergence means that we automatically treat different parameters differently!
  - If a change in one of the entries of ϕ results in large policy changes, then the constraint does not allow large changes in that entry.

- Since we want to have small updates (constrained by ϵ): approximation via the Fisher information matrix,

$$\phi' \leftarrow \arg\max_{\phi'} (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi) \qquad \text{subject to} \qquad (\phi' - \phi)^\top F (\phi' - \phi) \leq \epsilon,$$

$$\text{with} \qquad F = \mathbb{E}_{s\sim p_\phi, a\sim\pi_\phi(\cdot|s)}\left[(\nabla_\phi \log \pi_\phi(a|s))(\nabla_\phi \log \pi_\phi(a|s))^\top\right].$$

Natural policy gradient

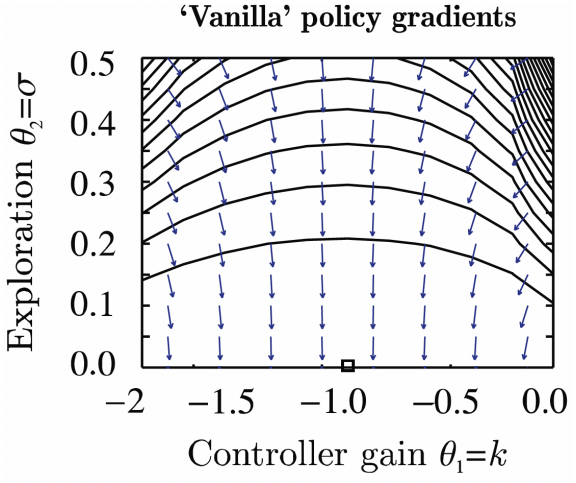
- We have now defined a policy gradient whose update length is constrained in terms of the KL divergence:

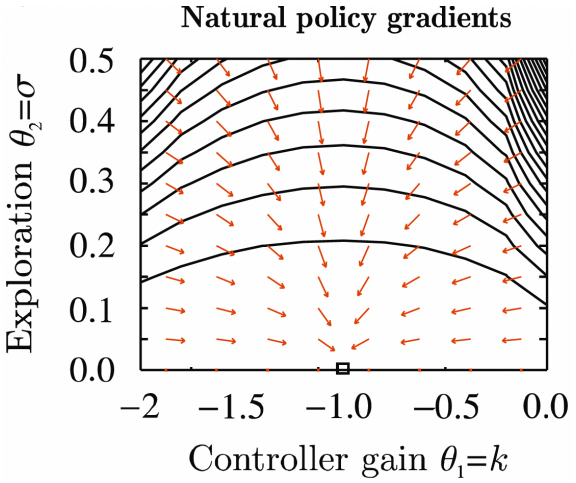
$$\phi' \leftarrow \arg\max_{\phi'} (\phi' - \phi)^\top \nabla_\phi L_\pi(\phi) \quad \text{subject to} \quad (\phi' - \phi)^\top F (\phi' - \phi) \leq \epsilon. \quad (3)$$

- What remains is the question of solving (3).
- Without going into further details (e.g., (S. M. Kakade 2001; Peters and Schaal 2008))
  $\Rightarrow$ Solution: **scale** the policy gradient **by the inverse Fisher information matrix**,

$$\phi \leftarrow \phi + \alpha F^{-1} \nabla_\phi L_\pi(\phi). \quad (4)$$

- **Intuition** behind the inverse FIM F^{-1}:
  - Parameters with a high impact have high Fisher information.
  - Scaling by the inverse "normalizes" the individual impacts.
- This techinque is known as the **natural policy gradient** (also **covariant policy gradient**).
  - Preconditioning steps of this type are very common in optimization in general.
  - Drawback: $F \in \mathbb{R}^{d \times d}$ can be very expensive to calculate (and invert) for very large parameter vectors.





Source: (Peters and Schaal 2008)

Limitations of the natural policy gradient

$$\phi \leftarrow \phi + \alpha F^{-1} \nabla_\phi L_\pi(\phi) \tag{4}$$

1. The step size α is hard to choose!

- Scaling doesn't prevent the step from changing the policy too much.
- Large steps can collapse performance.

2. No explicit control of policy change.

- NPG does not explicitly constrain the distance between π_{old} and π_{new}.
- Data for gradient estimation is sampled from π_{old}.
- If new policy differs too much, gradient estimate becomes unreliable.

3. Inversion of the Fisher matrix.

- NPG requires inversion of the Fisher Information Matrix.
- If our network has d parameters, the FIM is a $d \times d$ matrix.
- For a modest deep learning network with 100,000 parameters, F contains 10 billion elements.

4. Performance guarantees require small policy changes.

- NPG does not provide a mathematical guarantee that π_{new} will actually be better than (or at least as good as) π_{old}.
- Theory shows that small policy updates should improve performance (S. Kakade and Langford 2002) …
- … but the basic NPG update does not enforce the required small change condition.

- Storing this in memory and computing its exact inverse ($\mathcal{O}(d^3)$ complexity) $\Rightarrow$ computationally impossible.

Trust-region policy optimization

Performance measures of policies

- Let's introduce a new (and yet *already known* quantity): the **performance measure** of a policy,

$$\eta(\pi) = \mathbb{E}_{\tau \sim \pi} \left[\sum_{t=0}^{T-1} \gamma^t r_t \right] = \mathbb{E}_{s_0 \sim p_0} \left[V^{\pi}(s_0) \right].$$

- Remember our reinforcement learning objective

$$L_{\pi}(\phi) = \mathbb{E}_{\tau \sim p_{\phi}(\tau)} \left[\sum_{t=0}^{T-1} r_t \right] = \mathbb{E}_{s_0 \sim p_0} \left[V^{\pi_{\phi}}(s_0) \right] ?$$

- If we consider a policy parameterized by ϕ, i.e., π_{ϕ}, then the two equations are the same,

$$\eta(\pi_{\phi}) = L_{\pi}(\phi).$$

- The difference is more in terms of the *motivation*, not the definition:
  - We use $L_{\pi}(\phi)$ in policy gradients, since we want to find ascent directions for the network paraemters.
  - We use $\eta(\pi)$ as a global quantity to establish mathematical lower-bounds so that an algorithm can compute a safe step size (the trust region).

💡 For more details on this matter, see (S. Kakade and Langford 2002; Schulman et al. 2015).

Comparing the performance measure of two policies (1)

- Can we relate two policies π_{old} and π_{new} using advantage functions?
- Let's use $\gamma = 1$ for simplicity (but one can work this out with γ as well).
- Consider the advantage function under the old policy π_{old},

$$A_{\pi_{\text{old}}}(s_t, a_t) = r_t + V^{\pi_{\text{old}}}(s_{t+1}) - V^{\pi_{\text{old}}}(s_t).$$

- Now sample a trajectory under the new policy π_{new} (i.e., $(s_0, a_0) \to \ldots \to (s_{T-1}, a_{T-1})$) and calculate the advantages:

$$\begin{aligned}
A_{\pi_{\text{old}}}(s_0, a_0) &= r_0 + V^{\pi_{\text{old}}}(s_1) - V^{\pi_{\text{old}}}(s_0) \\
A_{\pi_{\text{old}}}(s_1, a_1) &= r_1 + V^{\pi_{\text{old}}}(s_2) - V^{\pi_{\text{old}}}(s_1) \\
&\vdots \\
A_{\pi_{\text{old}}}(s_{T-2}, a_{T-2}) &= r_{T-2} + V^{\pi_{\text{old}}}(s_{T-1}) - V^{\pi_{\text{old}}}(s_{T-2}) \\
A_{\pi_{\text{old}}}(s_{T-1}, a_{T-1}) &= r_{T-1} + \underbrace{V^{\pi_{\text{old}}}(s_T)}_{=0 \text{ (terminal)}} - V^{\pi_{\text{old}}}(s_{T-1})
\end{aligned}$$

Comparing the performance measure of two policies (2)

- Take the sum:

$$\sum_{t=0}^{T-1} A_{\pi_{\text{old}}}(s_t, a_t) = \underbrace{r_0 + V^{\pi_{\text{old}}}(s_1) - V^{\pi_{\text{old}}}(s_0)}_{=A_{\pi_{\text{old}}}(s_0, a_0)} + \underbrace{r_1 + V^{\pi_{\text{old}}}(s_2) - V^{\pi_{\text{old}}}(s_1)}_{=A_{\pi_{\text{old}}}(s_1, a_1)} + \ldots + \underbrace{r_{T-2} + V^{\pi_{\text{old}}}(s_{T-1}) - V^{\pi_{\text{old}}}(s_{T-2})}_{=A_{\pi_{\text{old}}}(s_{T-2}, a_{T-2})} + \underbrace{r_{T-1} - V^{\pi_{\text{old}}}(s_{T-1})}_{=A_{\pi_{\text{old}}}(s_{T-1}, a_{T-1})}$$

$$= r_0 + r_1 + \ldots + r_{T-2} + r_{T-1} - V^{\pi_{\text{old}}}(s_0) = \left(\sum_{t=0}^{T-1} r_t\right) - V^{\pi_{\text{old}}}(s_0).$$

- Shift around and take the expectation with respect to the new policy (in the case of $V^{\pi_{\text{old}}}$, this reduces to $s_0 \sim p_0$):

$$\mathbb{E}_{s \sim p_{\pi_{\text{new}}}, a \sim \pi_{\text{new}}}\left[\left(\sum_{t=0}^{T-1} r_t\right)\right] = \mathbb{E}_{s_0 \sim p_0}\left[V^{\pi_{\text{old}}}(s_0)\right] + \mathbb{E}_{s \sim p_{\pi_{\text{new}}}, a \sim \pi_{\text{new}}}\left[\sum_{t=0}^{T-1} A_{\pi_{\text{old}}}(s_t, a_t)\right]$$

$$\Leftrightarrow \qquad \eta(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \mathbb{E}_{s \sim p_{\pi_{\text{new}}}, a \sim \pi_{\text{new}}}\left[\sum_{t=0}^{T-1} \gamma^t A_{\pi_{\text{old}}}(s_t, a_t)\right]$$

- Right now, the expectation is written over an entire lifetime trajectory ($\tau \sim \pi_{\text{new}}$).
- To be useful for an optimization algorithm, we need to break it down step-by-step and look at individual states and actions.

Comparing the performance measure of two policies (3)

- Let's consider the continual case ($T = \infty$) such that we can disect the expectation into individual steps:

$$\begin{aligned}
\mathbb{E}_{s\sim p_{\pi_\text{new}}, a\sim\pi_\text{new}}\left[\sum_{t=0}^{\infty}\gamma^t A_{\pi_\text{old}}(s_t, a_t)\right] &= \sum_{t=0}^{\infty}\gamma^t \mathbb{E}_{s\sim p_{\pi_\text{new}}, a\sim\pi_\text{new}}\left[A_{\pi_\text{old}}(s_t, a_t)\right] \\
&= \sum_{t=0}^{\infty}\int_{\mathcal{S}} p_{\pi_\text{new}}(s)\int_{\mathcal{A}}\pi_\text{new}(a|s)A_{\pi_\text{old}}(s,a)\,\mathrm{d}a\,\mathrm{d}s \\
&= \int_{\mathcal{S}}\underbrace{\left(\sum_{t=0}^{\infty}\gamma^t p_{\pi_\text{new}}(s)\right)}_{=\rho_{\pi_\text{new}}(s)}\left(\int_{\mathcal{A}}\pi_\text{new}(a|s)A_{\pi_\text{old}}(s,a)\,\mathrm{d}a\right)\mathrm{d}s \\
&= \mathbb{E}_{s\sim\rho_{\pi_\text{new}}, a\sim\pi_\text{new}}\left[A_{\pi_\text{old}}(s,a)\right].
\end{aligned}$$

- Here, we have introduced the **discounted state distribution** $\rho_{\pi_\text{new}}(s)$.

Relation between the performance measures

$$\eta(\pi_\text{new}) = \eta(\pi_\text{old}) + \mathbb{E}_{s\sim\rho_{\pi_\text{new}}, a\sim\pi_\text{new}}\left[A_{\pi_\text{old}}(s,a)\right] \tag{5}$$

💡 When sampling, the data will automatically be distributed according to $\rho_{\pi_\text{new}}(s)$. See also the discussion in the actor-critic lecture.

Comparing the performance measure of two policies (4)

Relation between the performance measures

$$\eta(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \mathbb{E}_{s\sim\rho_{\pi_{\text{new}}},a\sim\pi_{\text{new}}}\left[A_{\pi_{\text{old}}}(s,a)\right] \tag{5}$$

What's so surprising about this equation?

- The Advantage function is built entirely out of the old policy's values,
- yet when we weight it by the new policy's state distribution,
- it perfectly calculates the new policy's total return.

Upside and downside of this identity

+ Proof that if you can find a new policy with a positive expected advantage under the old policy, your policy will get better.

— To compute the exact value, we must sample states from the new policy ($s \sim p_{\pi_{\text{new}}}$), which we do not yet know!

Solution approach in TRPO: "If π_{new} is very close to π_{old} (enforced by a KL-constraint), we can swap the state distribution $p_{\pi_{\text{new}}}$ for $p_{\pi_{\text{new}}}$ and safely optimize an approximation (i.e., a *surrogate objective*)."

How to turn this into an algorithm

Relation between the performance measures

$$\eta(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \mathbb{E}_{s\sim\rho_{\pi_{\text{new}}}, a\sim\pi_{\text{new}}}\left[A_{\pi_{\text{old}}}(s,a)\right] \tag{5}$$

Starting with (5), we need to take the several steps to arrive at an algorithm that we can actually implement.

1. Construct a local approximation of (5) (the *surrogate loss*) which we can estimate using samples.
2. Ensure that this approximation is sufficiently accurate.
3. Turn he surrogate loss function into a practical optimization problem.
4. Make sure that it scales to large problems.

Trust-region policy optimization (TRPO)
The first realization of these steps resulted in the TRPO algorithm, which was derived in (Schulman et al. 2015).

TRPO: The surrogate loss function

Relation between the performance measures

$$\eta(\pi_{\text{new}}) = \eta(\pi_{\text{old}}) + \mathbb{E}_{s\sim\rho_{\pi_{\text{new}}}, a\sim\pi_{\text{new}}}\left[A_{\pi_{\text{old}}}(s,a)\right] \tag{5}$$

- In principle, this equation shows us a way to guarantee policy improvement.
- If we can ensure positive expected advantages, then we improve!
- **But**: Sampling under the new policy ($s \sim \rho_{\pi_{\text{new}}}, a \sim \pi_{\text{new}}$) is infeasible, as we do not have it yet!

A surrogate loss function

- Simple trick: just sample the state s under the *old* policy!

$$L_{\pi_{\text{old}}}(\pi) = \eta(\pi_{\text{old}}) + \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}}, a\sim\pi}\left[A_{\pi_{\text{old}}}(s,a)\right]. \tag{6}$$

- Under the assumption that π_{old} and π_{new} do not differ too much, this is a reasonable assumption.
- In the limit case $\pi_{\text{old}} = \pi_{\text{new}}$, we have equality of (5) and (6), as well as their gradients (Schulman et al. 2015, Eq. (4)).

TRPO: Approximation error

In (Schulman et al. 2015), it was shown that error between the two expressions (5) and (6) can be bounded

$$\eta(\pi) \geq L_{\pi_{\text{old}}}(\pi) - C \cdot D_{\text{KL}}^{\max}(\pi_{\text{old}} \| \pi), \qquad \text{with} \quad D_{\text{KL}}^{\max}(\pi_{\text{old}} \| \pi) = \max_{s \in \mathcal{S}} D_{\text{KL}}(\pi_\phi(\cdot|s) \| \pi_{\phi'}(\cdot|s)),$$

and C is a constant derived from the environment's transition dynamics and the maximum bounds of the advantage function.

Central message: If we maximize the right-hand side of this inequality, we are guaranteed to improve (or at least maintain) our true performance $\eta(\pi)$!

Maximizing the surrogate loss $\Rightarrow$ introducing a parametric policy π_ϕ, our surrogate-based optimization problem is

$$\max_{\phi} L_{\pi_{\text{old}}}(\pi_\phi) - C \cdot D_{\text{KL}}^{\max}(\pi_{\text{old}} \| \pi_\phi),$$

or, if we consider two iterates ϕ and ϕ'

$$\max_{\phi'} L_{\pi_\phi}(\pi_{\phi'}) - C \cdot D_{\text{KL}}^{\max}(\pi_\phi \| \pi_{\phi'}).$$

Challenges:

1. The constant C is usually large and thus restrictive
   Solution: Transfer to a constraint with hyperparameter δ.

2. Computing the maximum KL divergence over all possible states ($D_{\mathrm{KL}}^{\max}\left(\pi_{\phi_{\text{old}}} \| \pi_{\phi}\right)$) is impossible.
Solution: Replace by the average KL divergence over the states that were actually observed:

$$\bar{D}_{\mathrm{KL}}\left(\pi_{\phi} \| \pi_{\phi_{\text{old}}}\right) = \mathbb{E}_{s \sim \rho_{\pi_{\phi_{\text{old}}}}}\left[D_{\mathrm{KL}}\left(\pi_{\phi_{\text{old}}}(\cdot|s) \| \pi_{\phi}(\cdot|s)\right)\right].$$

TRPO: Sampling the correct expectation

Our final observation is that in our surrogate loss function $L_{\pi_{\text{old}}}$ (Eq. (6)), the "old" performance measure $\eta(\pi_{\text{old}})$ serves as a constant $\Rightarrow$ we can neglect it!

- Overall, we obtain the following surrogate optimization problem:

$$\max_{\phi} \underbrace{\mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_\phi}[A_{\pi_{\text{old}}}(s,a)]}_{=L_{\pi_{\text{old}}}(\pi_\phi)-\eta(\pi_{\text{old}})} \quad \text{subject to} \quad \underbrace{\mathbb{E}_{s\sim\rho_{\pi_{\text{old}}}}[D_{\text{KL}}(\pi_{\text{old}}(\cdot|s)\|\pi_\phi(\cdot|s))]}_{=\bar{D}_{\text{KL}}(\pi_{\text{old}}\|\pi_\phi)} \leq \delta.$$

- What's wrong with this equation?
  $\Rightarrow$ In the loss, we're compute expectations over the new policy π_ϕ, but the data was collected under π_{old}!

$$\mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_\phi}\left[A_{\pi_{\text{old}}}(s,a)\right] = \int_{\mathcal{S}} \rho_{\pi_{\text{old}}}(s) \int_{\mathcal{A}} \pi_\phi(a|s) A_{\pi_{\text{old}}}(s,a)\,\mathrm{d}a\,\mathrm{d}s.$$

- **Solution**: Importance sampling!

$$\int_{\mathcal{S}} \rho_{\pi_{\text{old}}}(s) \int_{\mathcal{A}} \pi_{\text{old}}(a|s) \frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)} A_{\pi_{\text{old}}}(s,a)\,\mathrm{d}a\,\mathrm{d}s = \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\left[\frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)} A_{\pi_{\text{old}}}(s,a)\right].$$

TRPO: The surrogate optimization problem

Trust-region policy optimization (TRPO) optimization problem

$$\max_{\phi} \underbrace{\mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\left[\frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)}A_{\pi_{\text{old}}}(s,a)\right]}_{=L_{\text{TRPO}}(\phi)} \quad \text{subject to} \quad \underbrace{\mathbb{E}_{s\sim\rho_{\pi_{\text{old}}}}[D_{\text{KL}}(\pi_{\text{old}}(\cdot|s)\|\pi_\phi(\cdot|s))]}_{=\bar{D}_{\text{KL}}(\pi_{\text{old}}\|\pi_\phi)} \leq \delta. \quad (7)$$

Next question: How do we solve it efficiently? $\Rightarrow$ Let's start with the gradient!

$$\nabla_\phi L_{\text{TRPO}}(\phi) = \nabla_\phi \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\left[\frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)}A_{\pi_{\text{old}}}(s,a)\right] = \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\left[\frac{\nabla_\phi\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)}A_{\pi_{\text{old}}}(s,a)\right]$$

The Policy gradient for the TRPO surrogate loss

$$g = \nabla_\phi L_{\text{TRPO}}(\phi)\Big|_{\phi=\phi_{\text{old}}} = \mathbb{E}_{s\sim\rho_{\pi_{\phi_{\text{old}}}},a\sim\pi_{\phi_{\text{old}}}}\left[\frac{\nabla_\phi \pi_\phi(a|s)\big|_{\phi=\phi_{\text{old}}}}{\pi_{\phi_{\text{old}}}(a|s)} A_{\pi_{\phi_{\text{old}}}}(s,a)\right] \tag{8}$$

$$= \mathbb{E}_{s\sim\rho_{\pi_{\phi_{\text{old}}}},a\sim\pi_{\phi_{\text{old}}}}\left[\nabla_\phi \log \pi_\phi(a|s)\big|_{\phi=\phi_{\text{old}}} A_{\pi_{\phi_{\text{old}}}}(s,a)\right] \quad \textit{(Log-identity trick)}$$

is the standard policy gradient with value baseline!

TRPO: Solving the optimization problem (7)

To solve Problem (7) efficiently, we need two steps:

1. Linearize the objective function: First-order Taylor series expansion around ϕ_{old} using the gradient g (Eq. (8)).

$$L_{\text{TRPO}}(\phi) \approx L_{\text{TRPO}}(\phi_{\text{old}}) + g^\top(\phi - \phi_{\text{old}}) = g^\top(\phi - \phi_{\text{old}}).$$

   (The term $L_{\text{TRPO}}(\phi_{\text{old}})$ drops out because it is exactly zero at ϕ_{old}: No advantage and sampling ratio is one.)

2. Quadratic approximation of the KL constraint for small $\Delta\phi = (\phi - \phi_{\text{old}}) \Rightarrow$ Fisher information matrix (Eq. (2))!

Linear-quadratic approximation of the TRPO problem (7)

$$\max_{\Delta\phi} g^\top \Delta\phi \quad \text{subject to} \quad \frac{1}{2}\Delta\phi^\top F \Delta\phi \leq \delta, \tag{9}$$

where, according to (1),

$$F(\phi) = \mathbb{E}_{s\sim\rho_{\pi_{\phi_{\text{old}}}}, a\sim\pi_{\phi_{\text{old}}}}\left[\left[(\nabla_\phi \log \pi_\phi(a|s))(\nabla_\phi \log \pi_\phi(a|s))^\top\right]_{\phi=\phi_{\text{old}}}\right].$$

Solution to (9)

The solution (via Lagrange multipliers) yields

$$\Delta\phi = \sqrt{\frac{2\delta}{g^\top F^{-1} g}} F^{-1} g. \tag{10}$$

Note: This is NPG (4) with automatic step size selection $\alpha = \sqrt{\frac{2\delta}{g^\top F^{-1} g}}$!

💡 In classical optimization, the local approximation (9) is solved repeatedly. The solution can be shown to converge to the optimum of (7), which is known as **Sequential Quadratic Programming** (**SQP**). In TRPO, we solve the approximation (9) just once before collecting new data.

TRPO: Avoiding matrix inversion

$$\Delta\phi = \sqrt{\frac{2\delta}{g^\top F^{-1} g}} F^{-1} g \tag{10}$$

- Matrix inversion scales cubically. That is, for a d-dimensional parameter $\phi \in \mathbb{R}^d$, we have $\mathcal{O}(d^3)$.
- Solution: Use the conjugate gradient (CG) method for solving linear systems of the form $Fx = g$.
  - Usually, CG scales according to $\mathcal{O}(d^2)$ per iteration.
  - However, in TRPO, we can avoid calculating $F \in \mathbb{R}^{d \times d}$ entirely using automatic differentiation $\Rightarrow$ scales linearly with the number of weights (i.e., $\mathcal{O}(d)$).
  - This is as fast as it gets!

Approach

- Compute $x = F^{-1} g$ using the CG method ($\mathcal{O}(d)$ if we use autodiff).
- Calculate the denominator in (10): $g^\top F^{-1} g = g^\top x$.
- Compute the step size $\beta = \sqrt{\frac{2\delta}{g^\top x}}$.
- Compute the update $\Delta\phi = \beta x$.

TRPO: Line search

TRPO surrogate problem

$$\max_{\phi} L_{\text{TRPO}}(\phi) \quad \text{s.t.} \quad \bar{D}_{\text{KL}}\left(\pi_{\text{old}} \| \pi_{\phi}\right) \leq \delta. \qquad (7)$$

Linear-quadratic approximation

$$\max_{\Delta\phi} g^{\top}\Delta\phi \quad \text{subject to} \quad \frac{1}{2}\Delta\phi^{\top} F \Delta\phi \leq \delta, \qquad (9)$$

Issue: Even though we have selected an optimal step size for Problem (9) via Eq. (10), we are not guaranteed that in (7), we

- actually get a descent in $L_{\text{TRPO}}(\phi)$,
- satisfy the constraint $\bar{D}_{\text{KL}}\left(\pi_{\text{old}} \| \pi_{\phi}\right) \leq \delta$.

Step length selection via backtracking. Define a *shrinkage parameter* $\alpha \in (0, 1)$ and

1. Get $\Delta\phi$ by solving (9).
2. Calculate candidate $\phi' = \phi_{\text{old}} + \Delta\phi$ for the next iterate.
3. Check the following two conditions (recall that $L_{\text{TRPO}}(\phi_{\text{old}}) = 0$):

$$(I) \quad L_{\text{TRPO}}(\phi') > 0\,? \qquad \text{and} \qquad (II) \quad \bar{D}_{\text{KL}}\left(\pi_{\text{old}} \| \pi_{\phi'}\right) \leq \delta\,?$$

Note: Condition (II) is checked point-wise on the training batch:

$$\bar{D}_{\text{KL}}\left(\phi_{\text{old}} \| \phi_{\text{new}}\right) = \frac{1}{N}\sum_{i=1}^{N} D_{\text{KL}}\left(\pi_{\phi_{\text{old}}}(\cdot|s_i) \| \pi_{\phi_{\text{new}}}(\cdot|s_i)\right).$$

That's why TRPO does not apply to deterministic policies right away!

4. **if** (I) and (II) ✅ **then** $\phi_{\text{new}} \leftarrow \phi'$ and **STOP else:** $\Delta\phi \leftarrow \alpha\Delta\phi$ and back to 2.

TRPO: Final algorithm

Input: Initial policy parameters $\phi^{(0)}$, trust-region bound δ, backtracking parameter $\alpha \in (0,1)$, acceptance threshold $\epsilon > 0$.

For iterations $k = 0, 1, 2, \ldots$ until convergence:

1. **Data collection:** Run the current policy $\pi_{\phi^{(k)}}$ in the environment to collect a batch of trajectories $\mathcal{B} = \{(s_t, a_t, r_t)\}$.
2. **Advantage estimation:** Estimate the advantage values $A_{\theta^{(k)}}(s,a)$ for all sampled steps.
3. **Compute policy gradient (g):** Evaluate the standard policy gradient vector g using the collected samples:

$$g = \frac{1}{|\mathcal{B}|} \sum_{s,a \in \mathcal{B}} \nabla_\phi \log \pi_\phi(a|s)\Big|_{\phi=\phi^{(k)}} A_{\phi^{(k)}}(s,a)$$

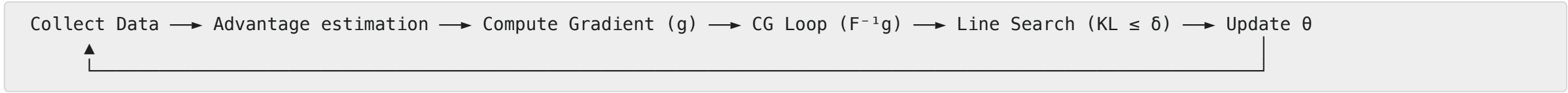
4. **Compute Step Direction (x) via Truncated CG** (10 to 20 iterations) to solve the linear system $Fx = g$ for $x \approx F^{-1}g$.
5. **Maximal step size (β):** Compute $\beta = \sqrt{\frac{2\delta}{x^T F x}}$. Set the full proposed step direction: $\Delta\phi = \beta x$.
6. **Backtracking:** Find largest factor $\gamma = \alpha^j$ (for $j = 0, 1, 2, \ldots$) such that $\phi_{\text{new}} = \phi^{(k)} + \gamma\Delta\phi$ satisfies:

$$L_{\text{TRPO}}(\phi_{\text{new}}) > 0 \qquad \text{and} \qquad \bar{D}_{\text{KL}}\left(\phi^{(k)} \middle\| \phi_{\text{new}}\right) \leq \delta$$

7. **Policy Update:** Update parameters to the safe, verified step: $\phi^{(k+1)} = \phi^{(k)} + \gamma\Delta\phi$ (or *reject* if no γ can be found).

TRPO: The big picture and relation to actor-critic

The Big Picture Structure



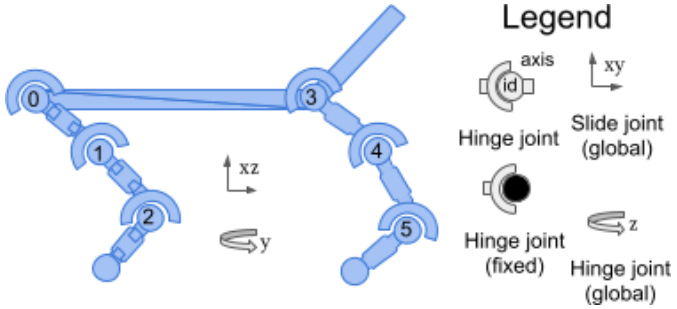
Relation to actor-critic

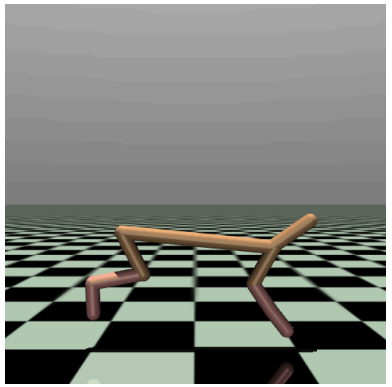
- At its core, TRPO is an Actor-Critic algorithm in practice:
  - It uses a value network critic to estimate $A_{\theta^{(k)}}(s, a)$.
- However, it was derived from first principles using policy gradient theory.
  - The primary goal of the paper was to solve a massive mathematical flaw in early actor-critic methods: the lack of monotonic improvement guarantees.

Example: Half-Cheetah

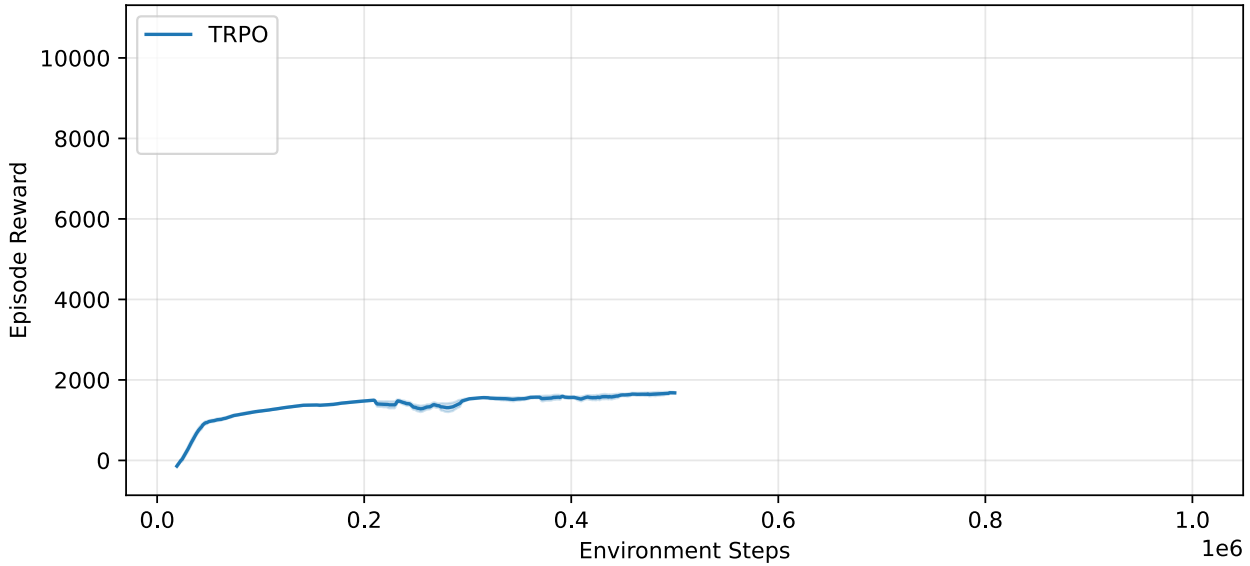
As an example, we study the Half Cheetah example from the MuJoCo library.

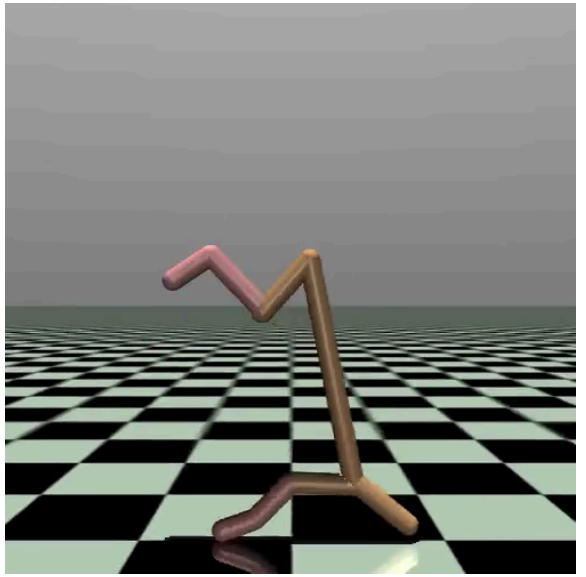
- $\mathcal{S}$: 17 states.
- $\mathcal{A}$: 6 actions.
- Reward: Forward-Cost - Control-Cost.





Results with TRPO (from the RL Baselines3 Zoo)





Proximal policy optimization (PPO)

Drawbacks of TRPO

1. Large computational overhead

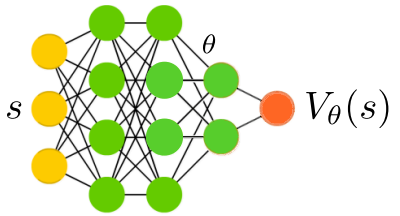
- TRPO relies on the Conjugate Gradient (CG) method to compute the matrix-free step direction $F^{-1}g$.
- While CG is much cheaper than explicit matrix inversion, it still requires running an inner loop for every single policy update, evaluating two backpropagation passes per CG step.

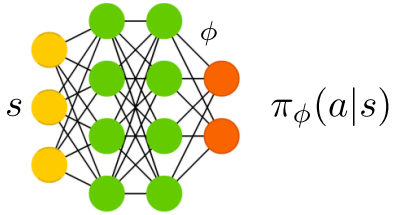
2. Code complexity and fragility

- TRPO requires custom optimization pipelines. You cannot simply hand a TRPO objective to a standard deep learning optimizer like Adam.
- Requires writing both a dedicated Conjugate Gradient solver and backtracking line search.

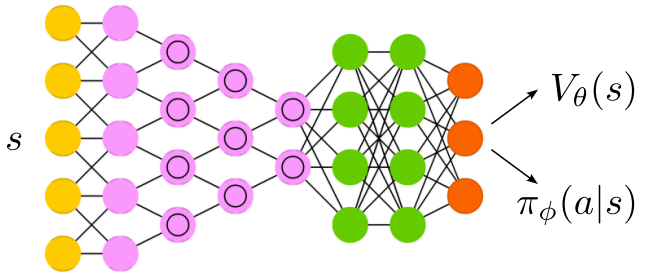
3. Incompatibility with shared architectures

- In modern deep RL, it is common to use a shared network architecture with two heads for the policy actions (Actor) and the state-value estimates (Critic).
- Because TRPO's constraint is calculated purely using the Fisher Information Matrix of the policy, it has no mathematical way to





Two separate networks



A shared network (e.g., a shared trunk and separate heads)

Inspired by Sergey Levine's CS285 lecture.

constrain or properly scale the value function parameters.

$\Rightarrow$ we are forced to maintain two entirely separate neural networks.

Avoiding the constraint

Breakthrough leading to **Proximal Policy Optimization** (**PPO**) (Schulman et al. 2017):
⇒ no need for an expensive constraint boundary for safe updates.
⇒ we can enforce a trust region inside the objective function by penalizing too large changes in the policy.

The clipped surrogate objective in PPO

$$L_{\text{CLIP}}(\phi) = \hat{\mathbb{E}}_t \left[\min \left(\kappa_t(\phi) A_t,\ \text{clip}(\kappa_t(\phi), 1-\epsilon, 1+\epsilon) A_t \right) \right], \qquad (11)$$

where we have simplified the notation in several ways:

- $\kappa_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{\text{old}}}(a_t|s_t)}$ is the *importance sampling* ratio.
- A_t: *estimated advantage* at time t, i.e., $A_\theta(s_t, a_t)$ (typically using a *critic*).
- ϵ is a hyperparameter (typically 0.1 or 0.2) determining how far the policy is allowed to drift.
- $\hat{\mathbb{E}}_t \left[f_t \right] = \frac{1}{T} \sum_{t=1}^{T} f_t$ denotes an *empirical expectation* (i.e., a sample average) which, in the limit $T \to \infty$, yields our analytical expectation $\mathbb{E}_{s \sim \rho_{\pi_\phi}, a \sim \pi_\phi} \left[f_t \right]$.

💡 This notation was chosen in (Schulman et al. 2017) for two reasons:

1. It aligns closely with the implementation.

Recall: The TRPO optimization problem (Eq. (7)) for comparison:

$$\max_\phi \mathbb{E}_{s \sim \rho_{\pi_{\text{old}}}, a \sim \pi_{\text{old}}} \left[\kappa(\phi) A_{\pi_{\text{old}}}(s,a) \right]$$

subject to

$$\mathbb{E}_{s \sim \rho_{\pi_{\text{old}}}} \left[D_{\text{KL}} \left(\pi_{\text{old}}(\cdot|s) \| \pi_\phi(\cdot|s) \right) \right] \leq \delta.$$

or, in short:

$$\max_\phi L_{\text{TRPO}}(\phi)$$
$$\text{s.t.} \quad \bar{D}_{\text{KL}} \left(\pi_{\text{old}} \| \pi_\phi \right) \leq \delta.$$

⇒ **clipping replaces the constraint!**

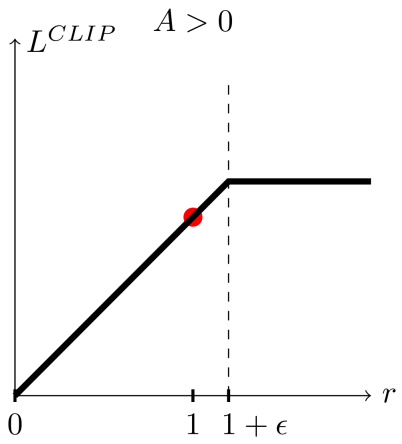
2. It handles both infinite and finite-time trajectories.

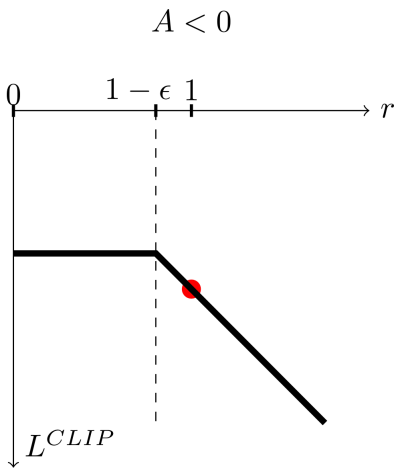
Understanding the clipped surrogate objective (1)

$$L_{\text{TRPO}}(\phi) = \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\Big[\underbrace{\frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)}}_{=\kappa(\phi)} A_{\pi_{\text{old}}}(s,a)\Big] \quad \text{vs.} \quad L_{\text{CLIP}}(\phi) = \hat{\mathbb{E}}_t\left[\min\left(\kappa_t(\phi)A_t,\ \text{clip}(\kappa_t(\phi), 1-\epsilon, 1+\epsilon)A_t\right)\right].$$

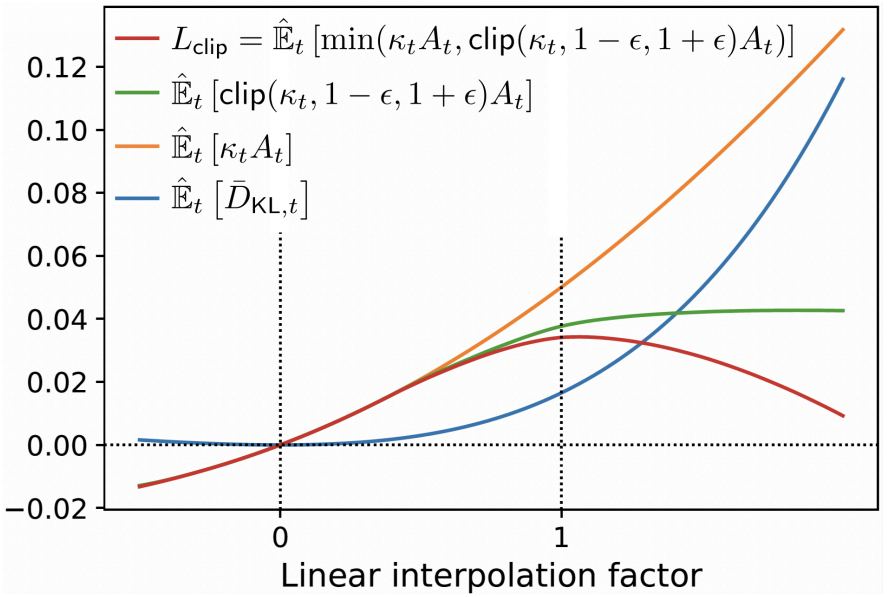
The clipping function $\text{clip}(r_t(\phi), 1-\epsilon, 1+\epsilon)$ bounds the importance sampling ratio κ_t to

- at most $1+\epsilon$ if $A > 0$,
- at least $1-\epsilon$ if $A < 0$.





The $\min$ **function** ensures that we take the minimum of the clipped and unclipped objective $\Rightarrow L_{\text{CLIP}}$ is a lower (i.e., pessimistic) bound on the unclipped objective.



Source: (Schulman et al. 2017)

Loss function behavior for interpolating policy $\pi_\alpha = \alpha\pi_\phi + (1-\alpha)\pi_{\text{old}}$. (Source: (Schulman et al. 2017))

Understanding the clipped surrogate objective (2)

$$L_{\text{TRPO}}(\phi) = \mathbb{E}_{s\sim\rho_{\pi_{\text{old}}},a\sim\pi_{\text{old}}}\Big[\underbrace{\frac{\pi_\phi(a|s)}{\pi_{\text{old}}(a|s)}}_{=\kappa(\phi)} A_{\pi_{\text{old}}}(s,a)\Big] \quad \text{vs.} \quad L_{\text{CLIP}}(\phi) = \hat{\mathbb{E}}_t\left[\min\left(\kappa_t(\phi)A_t,\ \text{clip}(\kappa_t(\phi), 1-\epsilon, 1+\epsilon)A_t\right)\right].$$

Examples

1. **Positive Advantage ($A_t > 0$)**

- This means the action was good, and the optimizer wants to make it more likely by increasing $\kappa_t(\phi)$ above 1.0.
- However, the clipping term caps the reward at $1+\epsilon$.
- The $\min$ operator ensures that even if $\kappa_t(\phi)$ grows (for instance, to 1.5 or 2.0), the optimizer receives no extra gradient incentive to push it further.
- It flattens the objective landscape, stopping the policy from overshooting.

2. **Negative Advantage ($A_t < 0$)**

- This means the action was bad, and the optimizer wants to make it less likely by dropping $\kappa_t(\phi)$ below 1.0.
- The clipping term caps this at $1-\epsilon$.
- **Important**: if the policy makes a very large change that makes a good action suddenly zero probability ($\kappa_t \to 0$), the objective drops significantly, keeping the policy in the safe zone.

PPO: Final algorithm

Input: Initial policy parameters $\phi^{(0)}$, clipping parameter ϵ, gradient descent algorithm (Adam, RMSprop, ...)

For iterations $k = 0, 1, 2, \ldots$ until convergence:

1. **Data collection:** Run the current policy $\pi_{\phi^{(k)}}$ in the environment to collect a batch of trajectories $\mathcal{B} = \{(s_t, a_t, r_t)\}$.
2. **Advantage estimation:** Estimate the advantage values $A_{\theta^{(k)}}(s, a)$ for all sampled steps.
3. **Optimize surrogate loss:** $\phi^{(k+1)} = \arg\min_{\phi} L_{\text{clip}}(\phi)$ via mini-batch training.

TRPO and PPO are on-policy!

- At first glance, TRPO and PPO can look off-policy because their objective functions use importance sampling:

$$\kappa_t(\phi) = \frac{\pi_\phi(a_t|s_t)}{\pi_{\phi_{\text{old}}}(a_t|s_t)}.$$

- However, the intent behind it here is different:
  - In **off-policy RL**: $\pi_{\theta_{old}}$ could be a completely different policy from long ago.
  - In **TRPO/PPO**: $\pi_{\theta_{old}}$ is the immediate, exact current policy that just finished stepping through the environment with the policy we're trying to update.

The Mathematical Reason: Local Approximations

- As derived earlier, the surrogate objectives for both TRPO and PPO rely on a critical assumption: the states we use to calculate the loss must be sampled from a distribution that is nearly identical to the new policy's distribution ($\rho_{\pi_\phi} \approx \rho_{\pi_{\phi_{\text{old}}}}$).
- Because their mathematical bounds collapse if the data isn't fresh, TRPO and PPO are **strictly on-policy**. If we tried to feed TRPO or PPO a batch of off-policy data collected by an entirely different policy/network:
  - In TRPO, the true KL divergence would exceed the trust region δ, the backtracking line search would reject the step, and the algorithm would freeze.
  - In PPO, the importance ratio $\kappa_t(\phi)$ would land far outside the $(1-\epsilon, 1+\epsilon)$ safety window, the clipping mechanism would zero out the gradients, likely resulting in no learning at all.

The one small exception: "Near-policy" (multiple epochs)

If changes are small, then we can try to use data in multiple consecutive iterations. Tradeoff:

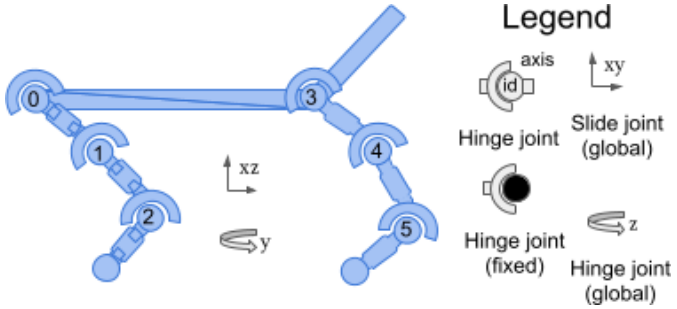
— Data is not entirely on-policy.

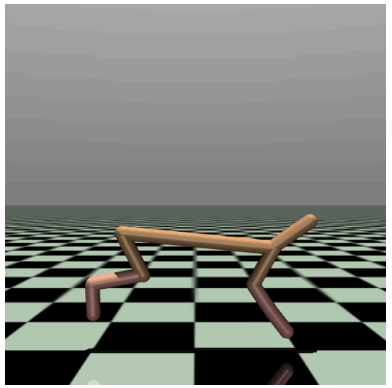
+ More data $\Rightarrow$ lower variance.

Example: Half-Cheetah

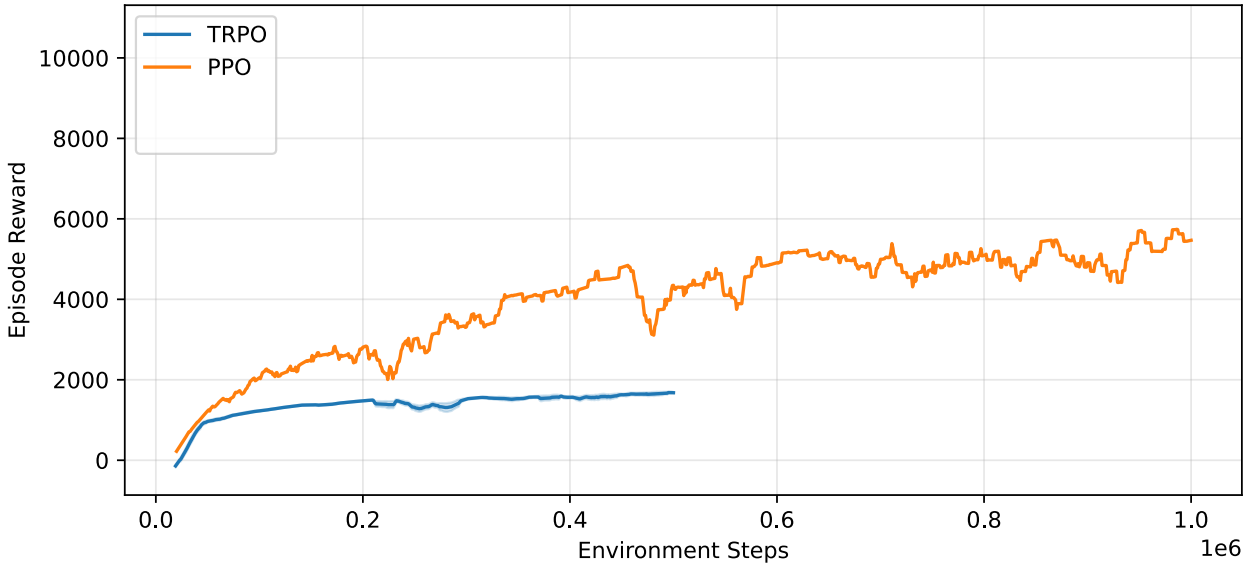
As an example, we study the Half Cheetah example from the MuJoCo library.

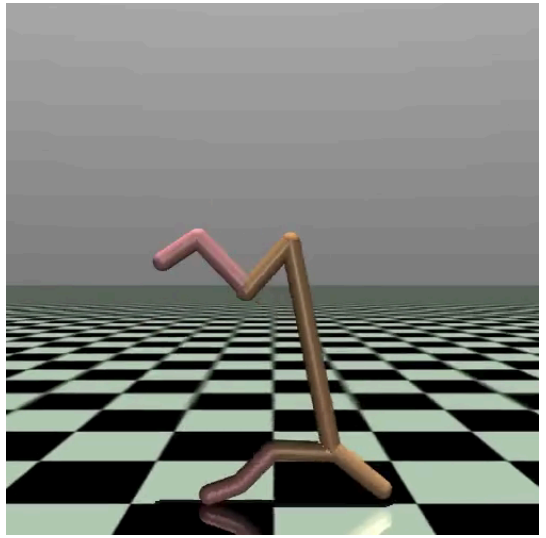
- $\mathcal{S}$: 17 states.
- $\mathcal{A}$: 6 actions.
- Reward: Forward-Cost - Control-Cost.



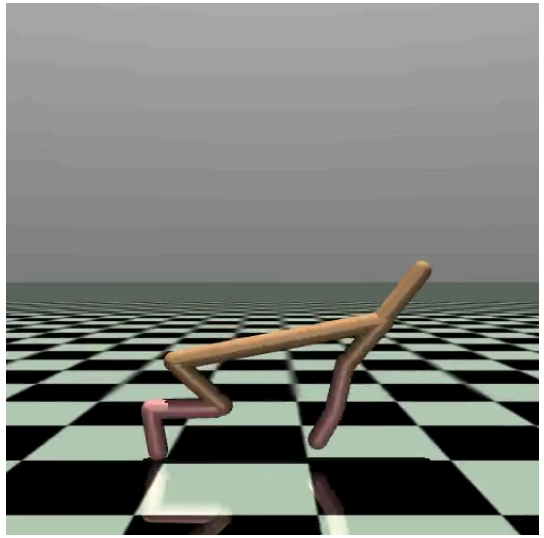


Results: TRPO vs. PPO (from the RL Baselines3 Zoo)





TRPO



PPO

Comparison of the algorithms we have seen in part I

Comparison of the algorithms we have seen in part I

- Standard policy gradient.
- Natural policy gradient ⇒ Balancing parameters via the Fisher information matrix.
- TRPO ⇒ Maximize surrogate objective subject to KL trust region.
- PPO ⇒ Replace hard trust region with clipped objective.

Feature	Natural Policy Gradient	TRPO	PPO
Core Concept	Scaled step sizes based on how much π changes, not ϕ	Strict boundary (trust region) to guarantee policy improvement	Mimics trust region via simplified clipped objective function
Mathematical Constraint	KL-divergence approximation via Fisher information matrix.	Strict: $\bar{D}_{\mathrm{KL}}\left(\pi_{\phi'} \| \pi_{\phi}\right) \leq \delta$	Soft constraint via clipped surrogate objective
Optimization Method	Matrix inversion: $\phi \leftarrow \phi + \alpha F^{-1} \nabla_{\phi} L_{\pi}(\phi)$	Conjugate Gradient (CG) + line search	Stochastic gradient descent (SGD) or Adam optimizer
Complexity	Very high (inversion of F)	High	Low (first-order optimization)
Implementation	Challenging	Very challenging	Simple
Data Efficiency	Poor	Moderate	High

References

Kakade, Sham M. 2001. “A Natural Policy Gradient.” In *Advances in Neural Information Processing Systems*, edited by T. Dietterich, S. Becker, and Z. Ghahramani. Vol. 14. MIT Press. https://proceedings.neurips.cc/paper_files/paper/2001/file/4b86abe48d358ecf194c56c69108433e-Paper.pdf.

Kakade, Sham, and John Langford. 2002. “Approximately Optimal Approximate Reinforcement Learning.” In *Proceedings of the Nineteenth International Conference on Machine Learning*, 267–74. ICML ’02. San Francisco, CA, USA: Morgan Kaufmann Publishers Inc.

Peters, Jan, and Stefan Schaal. 2008. “Natural Actor-Critic.” *Neurocomputing* 71 (7): 1180–90.

Schulman, John, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. 2015. “Trust Region Policy Optimization.” In *Proceedings of the 32nd International Conference on Machine Learning*, edited by Francis Bach and David Blei, 37:1889–97. Proceedings of Machine Learning Research. Lille, France: PMLR.

Schulman, John, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. “Proximal Policy Optimization Algorithms.” *arXiv:1707.06347*.